

Software Engineering Training Path (Web & Mobile)

TAQAT Academy– Professional Applied Program

Target Audience: High school graduates or diploma/bachelor holders with no prior technical background

Program Duration: 12 months

Program Structure: 3 Levels – **48 training hours per level** (Total: 144 hours)

Training Approach: 70% Practical – 30% Theoretical

Final Outcome: Junior Software Engineer (Web & Mobile) ready for the job market

Program Philosophy

This program is designed to produce **practical Software Engineers** capable of building real digital products across **web and mobile platforms**. The focus is on:

- Strong engineering fundamentals
- Clean and scalable code
- Full product thinking (front-end, back-end, mobile)
- Job readiness and teamwork

Level 1: Programming & Web Foundations (48 Hours)

Level Description

This level introduces learners to the fundamentals of programming and web development. It focuses on building a strong conceptual foundation in software engineering, computational thinking, and front-end web technologies, preparing participants for more advanced development tracks.

Level Objectives

By the end of this level, learners will be able to:

- Understand core programming concepts and logic
- Explain how software systems and web applications work
- Build static and basic responsive web pages
- Use essential developer tools and collaborate using version control
- Prepare for intermediate-level web and programming courses

Detailed Curriculum Breakdown (48 Hours)

Module 1: Introduction to Software Engineering (6 Hours)

Total Sessions: 3

Session 1 (2 Hours): Introduction to Software Engineering

Topics

- What is Software Engineering?
- Importance of software in modern systems
- Types of software:
 - Web applications
 - Mobile applications
 - Desktop applications

Activities

- Real-world examples discussion
- Identify software types used daily

Outcome

- Students understand what software engineering is and where it is used

Session 2 (2 Hours): Software Development Life Cycle (SDLC)

Topics

- SDLC overview
- Planning
- Design
- Development
- Testing
- Deployment
- Maintenance

Activities

- SDLC flow diagram
- Case study: building a simple app

Outcome

- Students understand how software is built from idea to production

Session 3 (2 Hours): Software Roles & Career Paths

Topics

- Frontend Developer
- Backend Developer
- Mobile Developer
- Full Stack Developer
- Career paths overview
- Industry roles and demand

Activities

- Career mapping exercise
- Q&A about job market

Outcome

- Students can identify suitable career paths in software engineering

Module 2: Programming Fundamentals with JavaScript (14 Hours)

Total Sessions: 7

Session 4 (2 Hours): Introduction to JavaScript

Topics

- What is JavaScript?
- Where JavaScript is used
- Running JS in browser & VS Code
- Console & basic syntax

Practice

- First JavaScript program

Session 5 (2 Hours): Variables & Data Types

Topics

- Variables (`var`, `let`, `const`)
- Data types:
 - Number
 - String
 - Boolean
 - Null & Undefined

Practice

- Simple calculations & logging output

Session 6 (2 Hours): Operators & Conditional Statements

Topics

- Arithmetic & logical operators
- Comparison operators
- `if`, `else`
- `switch`

Practice

- Decision-making programs

Session 7 (2 Hours): Loops

Topics

- `for` loop
- `while` loop
- Loop control (`break`, `continue`)

Practice

- Repetition-based exercises

Session 8 (2 Hours): Functions & Scope

Topics

- Function declaration
- Parameters & return values
- Scope (global vs local)

Practice

- Reusable function writing

Session 9 (2 Hours): Arrays & Objects

Topics

- Arrays
- Array methods
- Objects & properties
- Accessing data

Practice

- Small data-handling tasks

Session 10 (2 Hours): Error Handling & Coding Challenges

Topics

- Common JavaScript errors
- `try...catch`

- Debugging basics

Practice

- Mini coding challenges

Module 3: Web Fundamentals – HTML & CSS (10 Hours)

Total Sessions: 5

Session 11 (2 Hours): HTML Basics

Topics

- HTML structure
- Tags & attributes
- Headings, paragraphs, links, images

Practice

- Build a basic HTML page

Session 12 (2 Hours): Semantic HTML & Forms

Topics

- Semantic elements (`header`, `nav`, `section`, `footer`)
- Forms & input types
- Media elements

Practice

- Create a simple form page

Session 13 (2 Hours): CSS Fundamentals

Topics

- CSS syntax
- Colors, fonts
- Box model (margin, padding, border)

Practice

- Style HTML page

Session 14 (2 Hours): Layout with Flexbox & Grid

Topics

- Flexbox basics
- CSS Grid basics
- Page layout structure

Practice

- Build multi-section layout

Session 15 (2 Hours): Responsive Design

Topics

- Responsive design principles
- Media queries
- Mobile-first design

Practice

- Responsive webpage exercise

Module 4: Computational Thinking & Algorithms (8 Hours)

Total Sessions: 4

Session 16 (2 Hours): Computational Thinking Concepts

Topics

- What is computational thinking?
- Logical thinking
- Real-life problem solving

Practice

- Logic puzzles

Session 17 (2 Hours): Problem Decomposition & Pattern Recognition

Topics

- Breaking problems into smaller parts

- Identifying patterns

Practice

- Step-by-step problem analysis

Session 18 (2 Hours): Algorithms & Pseudocode

Topics

- What is an algorithm?
- Writing pseudocode
- Flowcharts

Practice

- Convert problems to pseudocode

Session 19 (2 Hours): Simple Algorithms Practice

Topics

- Searching basics
- Counting & sorting logic (conceptual)

Practice

- Algorithm exercises using pseudocode & JS

Module 5: Developer Tools & Collaboration (10 Hours)

Total Sessions: 5

Session 20 (2 Hours): VS Code & Project Structure

Topics

- VS Code interface
- Extensions
- Folder & file organization

Session 21 (2 Hours): Browser Developer Tools

Topics

- Inspect elements
- Console debugging

- Network tab basics

Practice

- Debugging simple issues

Session 22 (2 Hours): Git Basics

Topics

- What is Git?
- Repositories
- Commits
- Branches

Practice

- Local Git workflow

Session 23 (2 Hours): GitHub Basics

Topics

- Remote repositories
- Push & Pull
- GitHub interface

Practice

- Upload project to GitHub

Session 24 (2 Hours): Team Collaboration Workflow

Topics

- Collaboration basics
- Branching workflow
- Resolving conflicts
- Code review concept

Practice

- Simulated team workflow

Level 1 Practical Project

Project Description

Responsive Static Website

Learners will design and build a responsive static website, such as:

- Personal portfolio website
- Small business or company website

Project Requirements

- Use HTML5 semantic structure
- Apply CSS styling with Flexbox or Grid
- Ensure responsiveness across screen sizes
- Organize files properly
- Publish code to GitHub

Assessment & Evaluation

- Practical exercises during each module
- Participation and attendance
- Final project evaluation based on:
 - Code quality
 - Structure and responsiveness
 - Use of GitHub
 - Overall presentation

Level Outcomes

Upon successful completion of this level, learners will achieve:

- Solid understanding of programming fundamentals
- Clear knowledge of web structure and front-end basics
- Ability to build simple responsive websites
- Readiness to progress to **Level 2: Advanced Frontend / Backend Development**

Level 2: Application Development (Web & Mobile) (48 Hours)

Application Development Level

Level Description

This level focuses on building interactive web and mobile applications. Learners will move from static pages to dynamic, data-driven applications using modern front-end frameworks and cross-platform mobile technologies, while integrating real APIs and applying basic UI/UX principles.

Level Objectives

By the end of this level, learners will be able to:

- Build interactive web applications using modern JavaScript frameworks
- Develop cross-platform mobile applications
- Consume and integrate REST APIs
- Apply UI/UX and accessibility principles in development
- Prepare for junior-level front-end or mobile development roles

Prerequisites

- Successful completion of **Level 1: Programming & Web Foundations**
- Basic understanding of JavaScript, HTML, CSS, and Git

Detailed Curriculum Breakdown (48 Hours)

Session 1 (2 Hours): JavaScript Execution & Scope

Focus: How JavaScript runs under the hood

Topics

- JavaScript execution context
- Global vs function vs block scope
- Variable lifecycle
- Common scope-related bugs

Practice

- Predict output exercises
- Fix scope issues in sample code

Outcome

- Learners understand how JS code is executed and scoped

Session 2 (2 Hours): Closures, Hoisting & Modern JavaScript (ES6+)

Focus: Writing modern, clean JavaScript

Topics

- Hoisting behavior
- Closures (real-world use cases)
- `let` vs `const`
- Arrow functions
- Destructuring & template literals

Practice

- Convert ES5 code to ES6+
- Closure-based examples

Outcome

- Learners can use closures and ES6+ syntax confidently

Session 3 (2 Hours): DOM Manipulation

Focus: Making web pages dynamic

Topics

- DOM tree structure
- Selecting elements
- Creating, updating & removing elements
- Dynamic content rendering

Practice

- Build dynamic list / card UI

Outcome

- Learners can manipulate the DOM to update UI dynamically

Session 4 (2 Hours): Event Handling & User Interaction

Focus: Responding to user actions

Topics

- Event listeners
- Common event types
- Event bubbling & delegation (intro)
- Handling form inputs

Practice

- Interactive buttons & forms
- Event-driven UI behavior

Outcome

- Learners can build interactive user experiences

Session 5 (2 Hours): Asynchronous JavaScript Fundamentals

Focus: Understanding async behavior

Topics

- Synchronous vs asynchronous execution
- Callbacks & callback hell
- Promises
- Promise chaining

Practice

- Convert callbacks to promises

Outcome

- Learners understand async flow and promise-based logic

Session 6 (2 Hours): Async / Await & Fetch API

Focus: Working with real APIs

Topics

- Async / await syntax
- Fetch API
- Handling API responses
- Error handling (`try / catch`)

Practice

- Fetch data from a public API
- Display loading & error states

Outcome

- Learners can consume APIs and handle async operations properly

Session 7 (2 Hours): Mini Project – API-Driven Web Feature

Focus: Apply everything together

Topics

- API integration workflow
- Data rendering
- UI updates
- Code refactoring & cleanup

Practice

- Build a mini web app (e.g. users list, weather app, products gallery)

Outcome

- Learners can build a complete dynamic JS feature using APIs

Module Deliverables

- Multiple mini coding tasks
- One API-driven mini project
- Clean, structured JavaScript code

Module 2: Front-End Framework – React

Total Duration: 10 Hours

Session Length: 2 Hours

Total Sessions: 5

Session 8 (2 Hours): Introduction to React & Project Setup

Focus: Understanding React and starting a project

Topics

- What is React and why it's used
- SPA (Single Page Application) concept
- React vs traditional JavaScript
- Project setup (Create React App / Vite)
- Folder & file structure

Practice

- Create first React project
- Run and explore project structure

Outcome

- Learners understand React basics and project setup

Session 9 (2 Hours): Components & Props

Focus: Component-based architecture

Topics

- Functional components
- JSX syntax
- Props concept
- Passing data between components
- Component reuse

Practice

- Build reusable UI components
- Pass dynamic data using props

Outcome

- Learners can create and reuse components effectively

Session 10 (2 Hours): State Management & Side Effects

Focus: Making components dynamic

Topics

- State concept
- `useState` hook
- Updating UI based on state
- `useEffect` hook
- Component lifecycle basics

Practice

- Counter / data-fetching component
- State-driven UI updates

Outcome

- Learners can manage state and side effects in React

Session 11 (2 Hours): Events, Forms & Routing

Focus: User interaction and navigation

Topics

- Event handling in React
- Controlled components
- Forms & basic validation
- React Router
- Page navigation

Practice

- Build form with validation
- Multi-page React app with routing

Outcome

- Learners can handle user input and app navigation

Session 12 (2 Hours): Performance Basics & Mini React App

Focus: Putting everything together

Topics

- Basic performance concepts
- Re-rendering behavior
- Code organization best practices
- Mini React application structure

Practice

- Build a small React app:
 - Components
 - State
 - Routing
 - Forms

Outcome

- Learners can build a complete small React application

Module Deliverables

- Multiple React exercises
- One mini React application
- GitHub repository with clean structure

Module 3: Mobile Development – Flutter or React Native

Total Duration: 14 Hours

Session Length: 2 Hours

Total Sessions: 7

Session 13 (2 Hours): Introduction to Cross-Platform Mobile Development

Focus: Understanding the mobile development landscape

Topics

- Native vs cross-platform apps
- Why Flutter / React Native
- Real-world use cases
- Flutter vs React Native comparison

Practice

- Explore sample apps

Outcome

- Learners understand cross-platform mobile concepts and tool choices

Session 14 (2 Hours): Environment Setup & Project Structure

Focus: Getting ready to build apps

Topics

- Installing SDKs & tools
- Emulator / simulator setup
- Creating a new project
- Project folder structure

Practice

- Run first mobile app

Outcome

- Learners can set up and run a mobile project

Session 15 (2 Hours): Mobile UI Principles & Layout Basics

Focus: Designing for mobile screens

Topics

- Mobile UI/UX principles
- Responsive layouts
- Core layout components/widgets
- Styling basics

Practice

- Build simple UI screens

Outcome

- Learners can design basic mobile interfaces

Session 16 (2 Hours): Components & Layouts

Focus: Building structured mobile screens

Topics

- Core components/widgets
- Layout systems
- Reusable components
- UI composition

Practice

- Multi-section screen layout

Outcome

- Learners can structure mobile app UI effectively

Session 17 (2 Hours): Navigation & State Management Basics

Focus: App flow and data handling

Topics

- Navigation stacks
- Routing between screens
- Local state management
- Passing data between screens

Practice

- Multi-screen navigation app

Outcome

- Learners can manage navigation and simple state

Session 18 (2 Hours): API Integration & Async Handling

Focus: Connecting apps to backend services

Topics

- HTTP requests in mobile apps
- Fetching data from APIs
- Loading indicators
- Error handling

Practice

- API-driven mobile screen

Outcome

- Learners can integrate APIs into mobile apps

Session 19 (2 Hours): Mobile Mini Project

Focus: Apply everything together

Topics

- App structure planning
- UI refinement
- API integration
- Debugging & polishing

Practice

- Build a simple complete mobile app

Outcome

- Learners deliver a functional cross-platform mobile app

Module Deliverables

- Mobile UI screens
- API-integrated mobile app
- GitHub repository

Module 4: UI/UX Basics for Engineers

Total Duration: 10 Hours

Session Length: 2 Hours

Total Sessions: 5

Session 20 (2 Hours): Introduction to UI/UX Design

Focus: Understanding design from a developer's perspective

Topics

- What is UI vs UX
- Why UI/UX matters for engineers
- User-centered design mindset
- Good vs bad UI examples

Practice

- UI/UX critique of real applications

Outcome

- Learners understand the role of UI/UX in software quality

Session 21 (2 Hours): User Experience Principles

Focus: Designing for real users

Topics

- Core UX principles (clarity, consistency, feedback)
- User needs & expectations
- User personas
- User journeys

Practice

- Create persona & user journey for an app

Outcome

- Learners can analyze user behavior and needs

Session 22 (2 Hours): Accessibility Basics (WCAG Overview)

Focus: Inclusive design & accessibility

Topics

- What is accessibility?
- WCAG principles overview
- Color contrast & typography
- Keyboard navigation & screen readers

Practice

- Improve accessibility of an existing UI

Outcome

- Learners can apply basic accessibility improvements

Session 23 (2 Hours): Responsive & Mobile-First Design

Focus: Designing across devices

Topics

- Responsive design principles
- Mobile-first approach
- Layout patterns
- Cross-device testing

Practice

- Make UI responsive (web or mobile)

Outcome

- Learners can design responsive interfaces

Session 24 (2 Hours): Design-to-Code Workflow

Focus: Bridging design and development

Topics

- Reading Figma designs
- Design handoff basics
- Translating UI to code
- Common UX mistakes developers should avoid

Practice

- Convert Figma design into code-ready UI

Outcome

- Learners can accurately implement designs and avoid UX issues

Module Deliverables

- UX persona & journey
- Accessibility checklist
- Responsive UI example
- Design-to-code implementation

Level 2 Practical Project

Project Description

Functional Web Application OR Mobile Application

Learners will build either:

- An interactive web application using **ReactJs OR VueJs**
- A cross-platform mobile application using Flutter or React Native

Project Requirements

- Connect to a real or mock REST API
- Implement user interactions and state management
- Apply basic UI/UX principles
- Handle loading and error states
- Use GitHub for version control

Assessment & Evaluation

- Hands-on coding exercises
- Module-based mini tasks
- Final project evaluation based on:
 - Functionality
 - Code structure and cleanliness
 - API integration
 - UI/UX quality
 - GitHub usage

Level Outcomes

Upon successful completion of this level, learners will gain:

- Junior Front-End Developer skills
- Junior Mobile Developer skills
- Ability to build interactive, data-driven applications
- Practical experience working with APIs and modern frameworks

Level 3: Full Stack Engineering & Career Readiness

Duration: 48 Hours

Level: Professional Software Engineering

Format: Hands-on, project-based

Outcome: Job-ready Junior Software Engineer

Module 1: Backend Development Fundamentals (14 Hours)

Session 1 (2 Hours): Backend Foundations & Web Architecture

Topics

- Frontend vs Backend roles
- Client–Server Architecture
- HTTP & HTTPS basics
- Request–Response lifecycle
- REST principles overview

Practice

- Analyze real-world API requests (Postman / browser dev tools)

Assignment

- Explain backend flow for a real web app (diagram + short report)

Session 2 (2 Hours): Node.js / Laravel Environment Setup

Topics

- Node.js & Express OR Laravel structure
- Project initialization
- Environment variables
- Folder structure best practices

Practice

- Create a backend project scaffold

Assignment

- Push initialized backend project to GitHub

Session 3 (4 Hours): RESTful API Development

Topics

- REST API design
- Routes & Controllers
- Request validation
- API responses & status codes

Practice

- Build CRUD APIs for a resource (Users / Products)

Assignment

- Complete CRUD REST API with Postman collection

Session 4 (4 Hours): Authentication & Authorization

Topics

- Authentication vs Authorization
- JWT / Session-based auth
- Password hashing
- Role-based access control

Practice

- Login / Register API
- Protected routes

Assignment

- Secure API with auth middleware

Session 5 (2 Hours): MVC Architecture & Clean Code

Topics

- MVC pattern
- Controllers, Models, Services
- Clean code principles
- Error handling & logging

Assignment

- Refactor existing project to MVC structure

Module 2: Databases & Data Management (10 Hours)

Session 6 (2 Hours): Relational Databases Fundamentals

Topics

- MySQL / PostgreSQL overview
- Tables, rows, columns
- Primary & foreign keys
- Relationships

Practice

- Design simple ER diagram

Session 7 (2 Hours): Database Design & Normalization

Topics

- Database normalization
- One-to-One, One-to-Many, Many-to-Many
- Indexing basics

Assignment

- Design DB schema for capstone project

Session 8 (4 Hours): CRUD Operations & ORM

Topics

- SQL CRUD operations
- Using ORM (Sequelize / Eloquent)
- Migrations & seeders

Practice

- Connect backend to database
- Implement DB-backed APIs

Assignment

- Full CRUD API connected to database

Session 9 (2 Hours): Data Validation & Security

Topics

- Input validation
- SQL Injection prevention
- Data sanitization

Assignment

- Secure database queries & inputs

Module 3: Deployment & Software Quality (10 Hours)

Session 10 (2 Hours): Hosting, Domains & Environments

Topics

- Hosting types (Shared, VPS, Cloud)
- Domains & DNS
- Environment configs (Dev / Prod)

Session 11 (4 Hours): Deployment Basics

Topics

- Deploy backend to hosting (Vercel, Render, Railway, VPS)
- Database deployment
- Environment variables in production

Practice

- Deploy backend API live

Assignment

- Live deployed backend with public URL

Session 12 (2 Hours): Security Fundamentals

Topics

- HTTPS & SSL
- CORS
- API security best practices
- Rate limiting

Session 13 (2 Hours): Performance Optimization & Monitoring

Topics

- Query optimization
- Caching basics
- Logging & error tracking
- Scalability overview

Assignment

- Improve API performance & document changes

Module 4: Career & Professional Readiness (14 Hours)

Session 14 (3 Hours): Git & GitHub for Professionals

Topics

- Git workflow (branches, PRs)
- Writing clean commits
- GitHub project structure
- README & documentati

Assignment

- Professional GitHub repository setup

Session 15 (3 Hours): CV & LinkedIn for Engineers

Topics

- Technical CV structure
- Portfolio-based CVs
- LinkedIn optimization
- Personal branding

Assignment

- Publish optimized LinkedIn profile + CV

Session 16 (4 Hours): Technical Interviews & Coding Challenges

Topics

- Common backend interview questions
- System design basics (Junior level)

- Live coding interview practice
- Behavioral interview tips

Practice

- Mock technical interviews

Session 17 (4 Hours): Freelancing & Remote Work

Topics

- Freelancing platforms (Upwork, BrightGaza)
- Proposal writing
- Pricing & client communication
- Remote work tools & contracts

Assignment

- Create freelance profile + sample proposal

Capstone Project – Graduation Project

Project Requirements

- Full-stack application
- Web or Mobile frontend
- Backend API
- Database integration
- Authentication system
- Deployed live

Examples

- E-commerce system
- Booking platform
- Task management app
- Delivery or service marketplace

Evaluation Criteria

- Code quality
- Architecture
- Functionality
- Deployment
- Documentation
- Presentation

Assessment & Certification

- Weekly coding assignments
- Module-based evaluations
- Capstone project defense

Final Program Outcomes

Graduate Profile: Junior Software Engineer (Web & Mobile)

Capable of:

- Building full-stack applications
- Writing clean, maintainable backend code
- Working in teams using Git & Agile tools
- Applying for jobs, startups, or freelance platforms